DATA ENGINEERING WEEKLY

8

Window Functions, In Order



PART 2 OF THE START HERE SERIES

Running Sum #01

SUM(amount) OVER (PARTITION BY region ORDER BY date). The running total per region, up to and including the current row.



LAG & LEAD

#02

LAG(price, 1) and LEAD(price, 1), partitioned by ticker, put yesterday's and tomorrow's price on today's row. No self-join needed.



Month-over-Month

#03

Pull last month's revenue with a window function, compute growth %, then drop the first month: it has nothing to compare to.



Moving Average #04

AVG(reading) OVER (PARTITION BY sensor_id ORDER BY timestamp ROWS BETWEEN 2 PRECEDING AND CURRENT ROW). The row plus the 2 before it.



Top 3 With Ties #05

DENSE_RANK, not ROW_NUMBER. "Top 3 scores" can mean 5 players, and ROW_NUMBER would arbitrarily drop the tied ones.



QUALIFY Clause #06

Filter on a window result directly, no subquery: QUALIFY ROW_NUMBER() OVER (PARTITION BY customer_id ORDER BY purchase_time) = 1



Rows Are Not Days

#07

ROWS BETWEEN 6 PRECEDING is 7 rows. When stores skip days, that reaches weeks back. RANGE BETWEEN INTERVAL 6 DAY PRECEDING measures time instead.



Session Detection #08

LAG the previous event per user. A gap over 30 minutes starts a new session, and a running count of those starts is the session_id.



Save It.
Solve It.

All 8 are free on leetdata.ai, with
hints and an editorial.

leetdata.ai/problems

Part 3 lands next Tuesday.
Follow Data Engineering Weekly.